
¡Claude
Community
Colombia!
sonido

GRACIAS

Pontificia Universidad Javeriana

Facultad de Ingeniería



Ing. Francisco Calderón

Departamento de Electrónica

COMUNIDAD

El Parche de Claude

Grupo de WhatsApp de Claude Community Colombia

Eventos: **claudecolombia.com**



ANTES DE EMPEZAR

`platform.claude.com`

1. Entrar con el **correo de Luma**
2. Redimir los **créditos**
3. Abrir **Managed Agents**

SKILL DEL TALLER

agent-sdk-wizard

A plugin de Claude Code

```
$ claude plugin marketplace add JairoTorregrosa/jaiskills
$ claude plugin install jaiskills@jaiskills
claude › /jaiskills:agent-sdk-wizard
```

B npx skills

```
$ npx skills add JairoTorregrosa/jaiskills -s agent-sdk-wizard
claude › /agent-sdk-wizard
```

github.com/JairoTorregrosa/jaiskills
skills/models/agent-sdk-wizard

Claude Community Bogotá

Primer agente con la Claude Agent SDK

The Workshop

CLAUDE COMMUNITY

Colombia



Jairo Torregrosa

Sr AI Research Engineer, Mercado Libre · Claude Community Ambassador

LA PREGUNTA DE HOY

«¿Cuándo es mi próximo vuelo?»

De usar agentes a construirlos

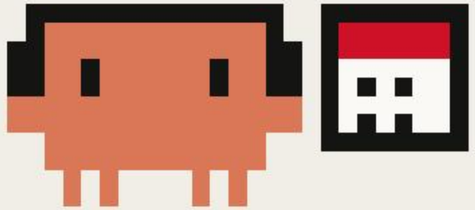


usuario de agentes



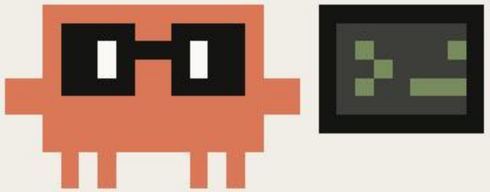
creador de agentes

Ideas de agentes



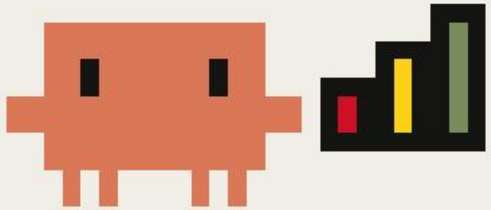
Asistente personal

agenda · correo · proyectos



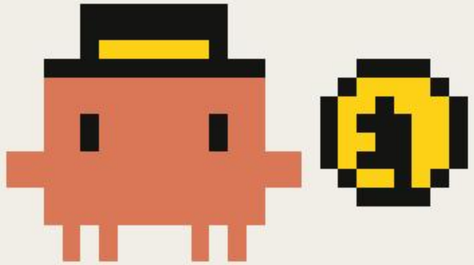
Código y QA

review · pruebas · DevOps



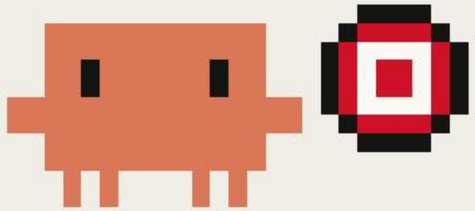
Datos

calidad · SQL · alertas



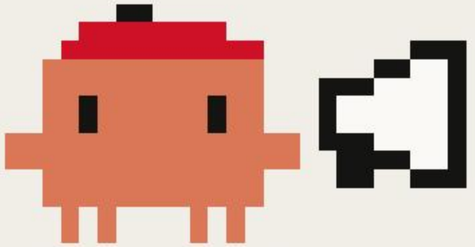
Finanzas

gastos · facturas · inversión



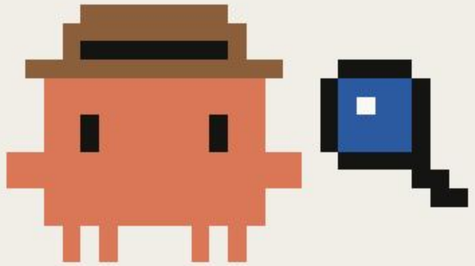
Ventas

leads · CRM · seguimiento



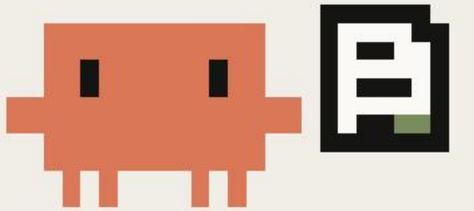
Marketing y contenido

redes · campañas · diseño



Investigación

noticias · papers · competencia



Documentos y atención

contratos · tickets · tutor

Agenda

9:10	 Crear un agente	créditos · Managed Agents
9:40	 Entender el loop	conceptos
10:10	 Actuar el loop	la sala es el agente
11:00	Pausa	café
11:15	 Práctica	wizard · Agent SDK
12:10	 Demos	ruleta · 4 agentes · una gorra
12:30	Cierre	foto · preguntas



BLOQUE 1

Un agente en diez minutos, sin código

Claude Managed Agents · el loop corre en Anthropic

▶ COMPARTO PANTALLA

Créditos de API en platform.claude.com

`mismo correo de la inscripción en Luma`

10 min

SKILL DEL TALLER

agent-sdk-wizard

A plugin de Claude Code

```
$ claude plugin marketplace add JairoTorregrosa/jaiskills
$ claude plugin install jaiskills@jaiskills
claude › /jaiskills:agent-sdk-wizard
```

B npx skills

```
$ npx skills add JairoTorregrosa/jaiskills -s agent-sdk-wizard
claude › /agent-sdk-wizard
```

github.com/JairoTorregrosa/jaiskills
skills/models/agent-sdk-wizard

▶ COMPARTO PANTALLA

Console: crear el agente, abrir una sesión, revisar Events

`platform.claude.com/workspaces/default/agent-quickstart`

15 min

Tres piezas definen el agente



system prompt

rol y reglas



tools

acciones disponibles



skills

procedimientos escritos



BLOQUE 2

Entender el loop

Qué ocurre dentro del agente recién creado

QUÉ ES UN AGENTE

Un agente tiene un objetivo y elige acciones para cumplirlo.

DEFINICIÓN CORTA · ANTHROPIC

agente =



LLM

+



tools

+



loop

«LLMs autonomously using tools in a loop»

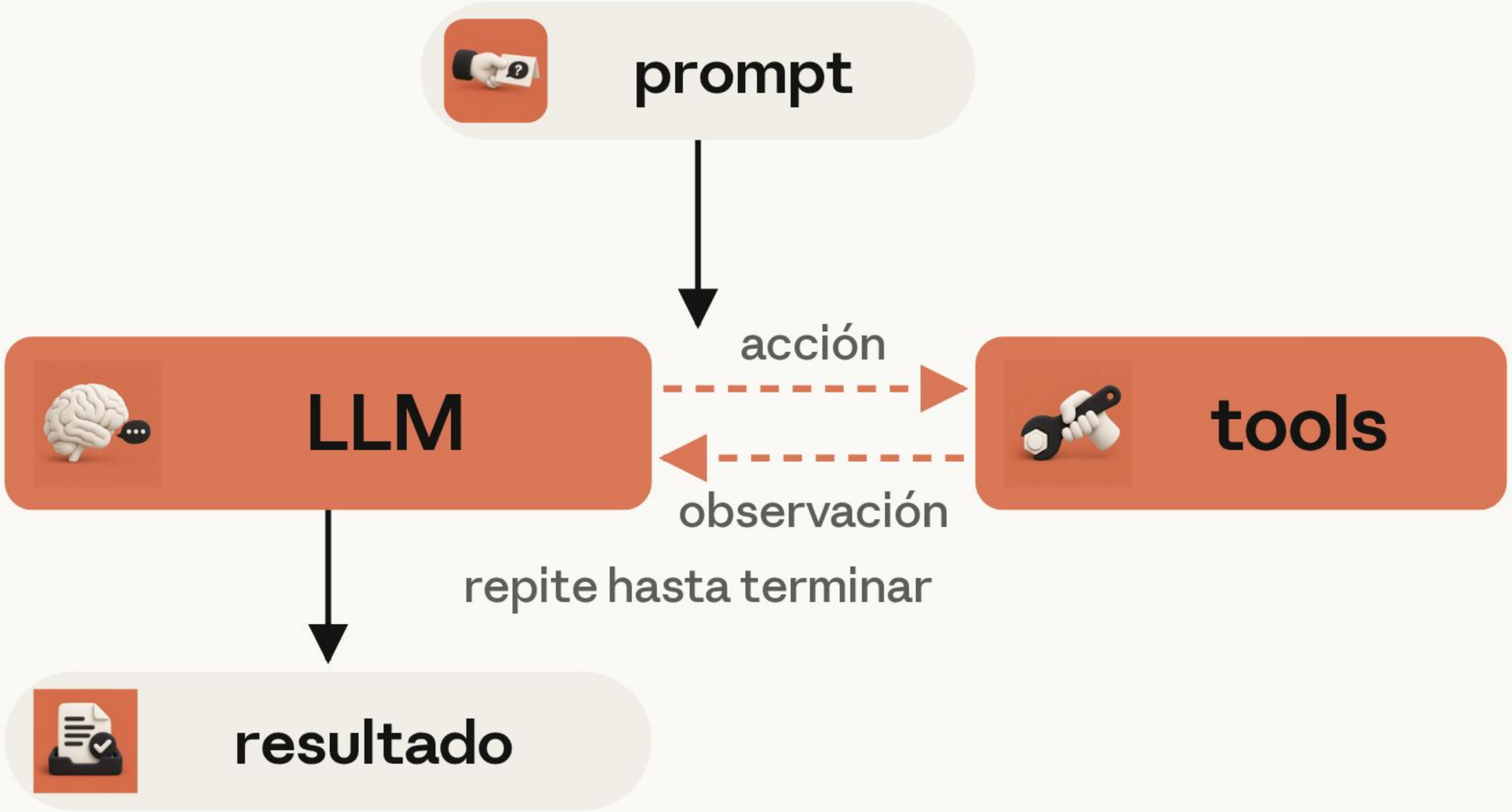
DEFINICIÓN DE TRABAJO

agente =  + 

LLM **harness**

harness: todo el código alrededor del LLM

El loop


prompt
LLM
acción
observación
tools
repite hasta terminar
resultado

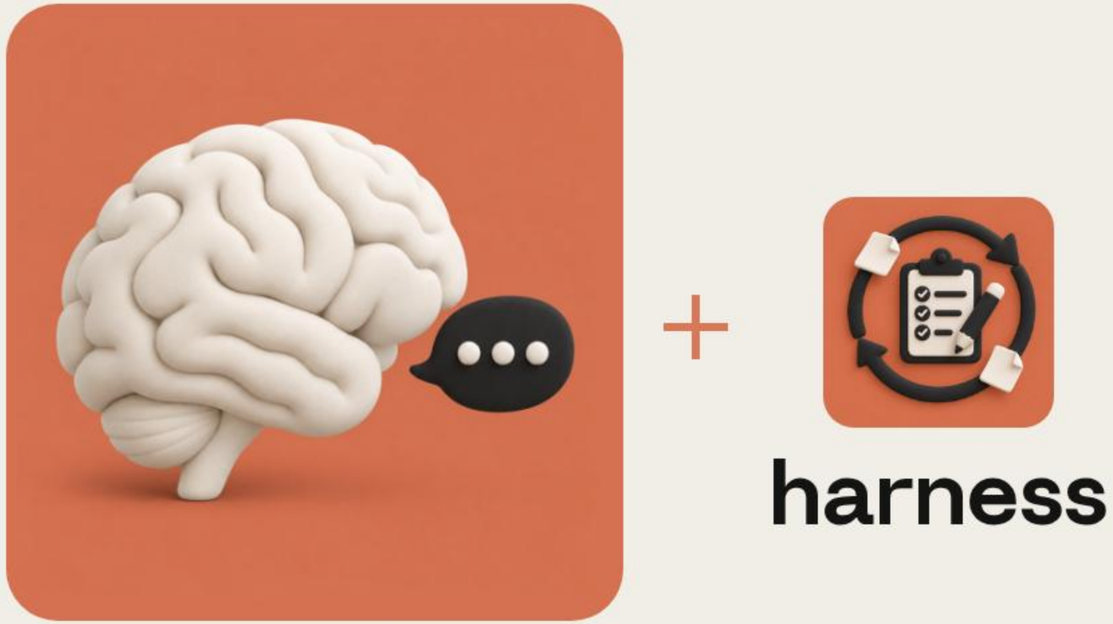
LLM pequeño, harness grande

LLM pequeño



LLM + harness

LLM capaz



LLM + harness

Cuando el LLM mejora, el harness cambia.

Un agente no necesita chat



un prompt



el loop corre solo



un resultado

`cron` `CI` `webhook` `script` `claude -p`

Ocho actores, cinco papeles



Usuario



Harness



LLM



Hook



Tool



Skill



MCP



Subagente



system prompt



user message



tool call



tool result



assistant message

Harness: recibe todo, reparte todo

ENTRA



user message



tool call



tool result



assistant message



el único que habla con el LLM y con el usuario

SALE



contexto → LLM



tool call → quien ejecuta



respuesta → usuario

LLM: lee el contexto, entrega un papel

ENTRA



todo el contexto



no ve ni toca nada:
lee y escribe

SALE



tool call



o assistant message

Hook: cada tool call pasa por aquí

ENTRA



tool call



código propio,
determinista

SALE



pasa

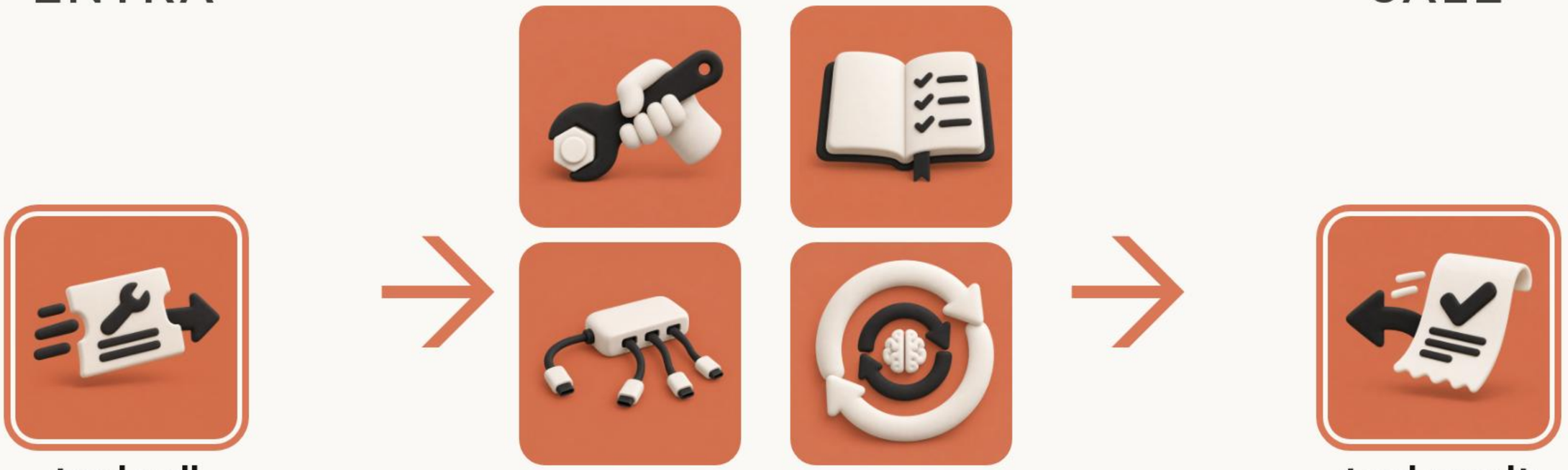


o error: no pasa

Quien ejecuta: recibe un tool call, devuelve un tool result

ENTRA

SALE



tool call

tool result

para el LLM los cuatro son lo mismo

Un loop, papel por papel

Otro loop: **assistant message**



Contexto

- Eres un asistente…
- ¿llueve hoy en Bogotá?
- #1 clima("Bogotá")
- #1 lluvia 80 %
- Sí: lleva sombrilla

Cuando el hook dice no

El LLM lee el error y cambia de plan



Contexto

#1 Bash("rm -rf repos/")

#1 ERROR · hook dijo no

#2 Bash("ls repos/")

¿Quién decide cuándo corre?

El LLM lo pide



tool



MCP



skill



subagente

puede pasar, o no

El harness lo obliga



hook



permisos



topes

pasa siempre

MENSAJE 2

Lo que no está en el contexto, el LLM no lo sabe.

MENSAJE 1

El LLM solo habla. El harness reparte, anota y decide cuándo parar.



BLOQUE 3

Actuar el loop

Ocho personas al frente. El resto anota la secuencia de papeles.

Ocho voluntarios, ocho letreros



Harness

pega cada papel en el contexto



LLM

de espaldas al contexto



Hook

entre el harness y los demás



Usuario

hace la pregunta



Tool

hoy()



Skill

buscar-vuelo



MCP

gmail: buscar · leer



Subagente

entra en la segunda ronda

USER MESSAGE

«Revisa mis correos de la última semana y dime cuándo es mi próximo vuelo.»

MCP: la bandeja de correo



clic en un correo = `gmail.leer`(id) → correo completo

Tool y skill disponibles



TOOL · CÓDIGO

hoy()

↳ devuelve

```
"sábado 7 de noviembre de 2026"
```

Corre y entrega un dato.



SKILL · TEXTO

buscar-vuelo

`SKILL.md`

1. Pedir la fecha de hoy — `hoy()`
2. Buscar correos de aerolíneas, últimos 7 días — `gmail.buscar`
3. Ignorar promociones y encuestas
4. Leer el correo completo — `gmail.leer`
5. Responder el vuelo futuro más cercano

El LLM la lee solo cuando la pide.

Segunda ronda: entra el subagente

Al contexto entra 1 papel en vez de 6



Contexto

```
Eres un asistente personal…
¿mi próximo vuelo?
#1 skill cargada · #2 sáb 7 nov
#3 Agent("busca el vuelo futuro…
#3 LATAM · jue 12 nov 06:15
```

Replay: así lo hizo el agente real

Recibo: 1 turno · 5 llamadas al LLM · USD 0.014



Contexto

```
buscar-vuelo: los 5 pasos
#2 hoy()
#2 sáb 7 nov 2026
#3 gmail.buscar(desde 31 oct)
#3 c1 · c2 · c3 · c4
#4 gmail.leer("c2")
#5 gmail.leer("c3")
#4 Wingo · 1 nov · ya pasó
#5 LATAM · jue 12 nov 06:15
LATAM · jue 12 nov · 06:15
```

PARA LA PAUSA

Claude Code ya hace todo esto. Entonces, ¿para qué un SDK?



BLOQUE 4 · PRÁCTICA

Un agente propio, con la Claude Agent SDK

el wizard pregunta, escribe el código y lo corre

⏳ abrir el timer · hasta 12:10

▶ abrir el demo

El mismo loop, en tres lugares



Claude Code

corre en la terminal



Agent SDK

corre dentro de un programa



Managed Agents

corre en Anthropic

¿Para qué un SDK?

1 Lo probado a mano **corre solo**: cron, CI, bot.

2 El harness de Claude Code, **dentro de un producto**.

3 Agentes **armados desde código**: datos, paralelo, reglas propias.

SKILL DEL TALLER

agent-sdk-wizard

A plugin de Claude Code

```
$ claude plugin marketplace add JairoTorregrosa/jaiskills
$ claude plugin install jaiskills@jaiskills
claude › /jaiskills:agent-sdk-wizard
```

B npx skills

```
$ npx skills add JairoTorregrosa/jaiskills -s agent-sdk-wizard
claude › /agent-sdk-wizard
```



github.com/JairoTorregrosa/jaiskills
skills/models/agent-sdk-wizard

Tres formas de autenticar el agente

A · RECOMENDADO

`ant auth login`

Sesión de platform. Sin keys.

B

`ANTHROPIC_API_KEY`

Key en **.env**. Producción.

C

`claude setup-token`

Pro / Max. Uso personal, sin créditos.

Demo de 4 minutos: qué mostrar



System prompt

rol · reglas · formato



Tools

propias · MCP



Skills

procedimiento



Subagentes

si aplica



Agente corriendo

en vivo

⏳ `timer de práctica`

MENSAJE 4

El loop ya existe. El trabajo está en el system prompt, las tools y los guardrails.



12:10 : DEMOS

Ruleta: cuatro agentes, una gorra

abrir la ruleta

GRACIAS, ANTHROPIC · GRACIAS, JAVERIANA · GRACIAS, BOGOTÁ

De usuarios a creadores de agentes

Próximos eventos: El Parche de Claude

Skill:
github.com/JairoTorregrosa/jaiskills

